

HIRELOOP

India-first AI recruiting platform

Two AI agents. One candidate graph. Built for the Indian job market.

[Build overview](#) · [Architecture](#) · [Status](#) · [Scale](#)

Prepared for team review · June 2026



What we're building

Hireloop is a two-sided, AI-run recruiting platform for India. Candidates are guided end-to-end by Aarya — an AI career partner that ingests their LinkedIn, CV and a short voice call, then surfaces real, scored job matches. Recruiters are served by Nitya, which drafts warm intros and runs their pipeline. A shared Postgres candidate graph connects both sides.

Aarya

candidate AI agent

Nitya

recruiter AI agent

23

build steps mapped

8

steps built

30+

database tables

12

external tools / APIs

A two-sided marketplace, run by AI



CANDIDATE SIDE

Aarya

- Signup via LinkedIn
- Auto-built profile (LinkedIn + CV)
- 15-min voice / text career chat
- Scored, explained job matches
- One-tap "Request intro"
- Tailored résumé + mock interview



Shared candidate graph

Supabase Postgres + pgvector
embeddings · RLS · realtime



RECRUITER SIDE

Nitya

- Recruiter signup + role intake
- Nitya generates the hiring brief
- Per-role candidate search
- Kanban pipeline of candidates
- Warm intros drafted + Gmail-sent
- Inbox + status handshake

Two agents on one LangGraph runtime



Aarya

Candidate-facing career partner

TOOLS

- profile_read
- build_career_path
- job_search
- get_match_score
- request_intro
- save_job / direct_apply



Nitya

Recruiter & hiring-manager agent

TOOLS

- candidate_lookup
- draft_email
- send_via_gmail
- update_intro_status
- pipeline ops
- role brief generation

Agent runtime

LangGraph 1.x

single-threaded master loop, Postgres checkpoints

OpenRouter · Claude

LLM reasoning + tool-calling

Deepgram → Sarvam

voice STT + TTS (Sarvam planned, Indian langs)

agent_actions

every tool call → table → Realtime UI counter

The full stack, by layer



Frontend

Next.js 15 · React 18 · TypeScript (strict) · Tailwind · Radix UI · Zustand · React-Hook-Form + Zod · Framer Motion



Backend API

FastAPI · Python 3.12 · Pydantic v2 · asyncpg · httpx · structlog · Uvicorn · python-jose



AI / Agents

LangGraph 1.x · langchain-openai · OpenRouter (Claude) · Deepgram STT/TTS now → Sarvam AI planned (Indian langs)



Data

Supabase Postgres · pgvector (HNSW, cosine) · Row-Level Security · Realtime · Storage buckets · pg_cron



External services

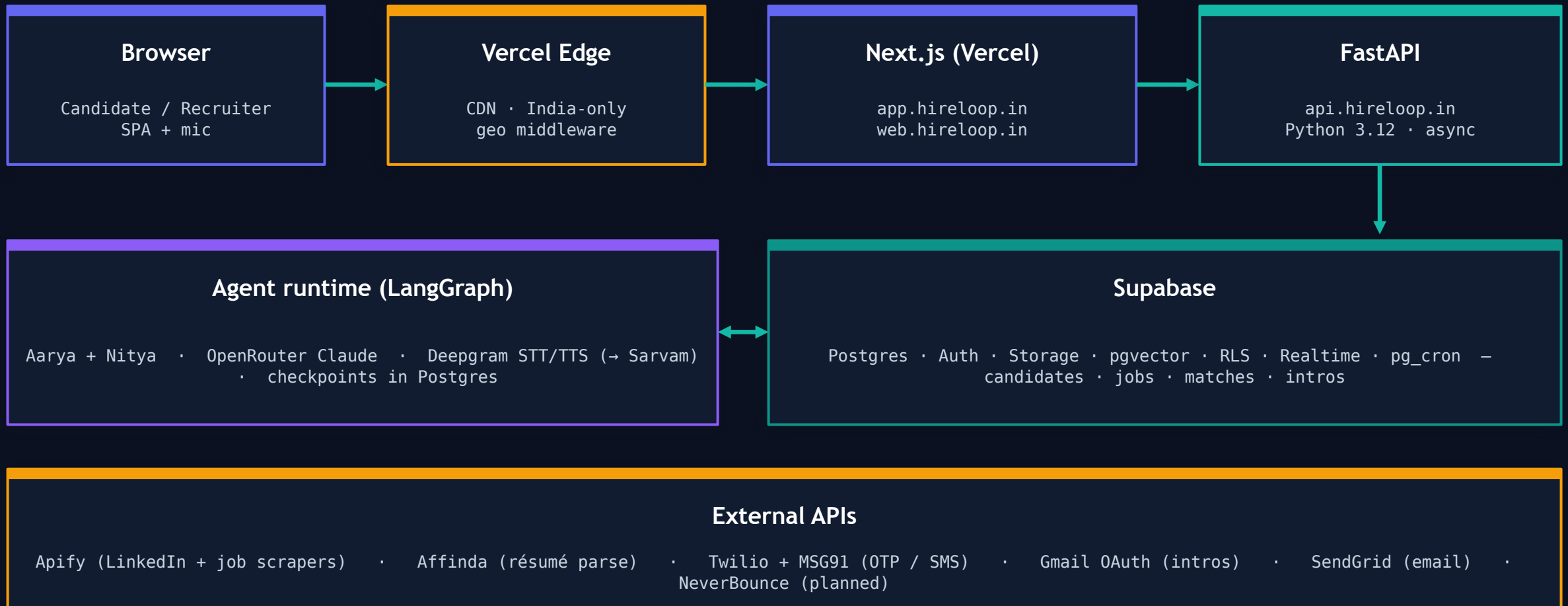
Apify (LinkedIn + jobs) · Affinda (résumé) · Twilio + MSG91 (OTP / SMS) · Gmail OAuth (intros) · SendGrid (email) · NeverBounce (planned)



Hosting & infra

Vercel (app + web · edge CDN + India geo) · Supabase (Postgres · Auth · Storage · Realtime) · GitHub Actions CI/CD

How requests flow, end to end



30+ tables across six domains



Identity & consent

users
candidates
recruiters
consent_log
dpdp_export_jobs



Candidate profile

resumes
career_paths
candidate_embeddings
voice_sessions
saved_jobs



Jobs & matching

jobs
companies
job_embeddings
match_scores
match_audits
job_ingest_log



Intros & hiring

intro_requests
hiring_managers
gmail_tokens
job_applications
placements



Comms & interviews

conversations
messages
agent_actions
mock_interviews
notifications
whatsapp_messages
tailored_resumes

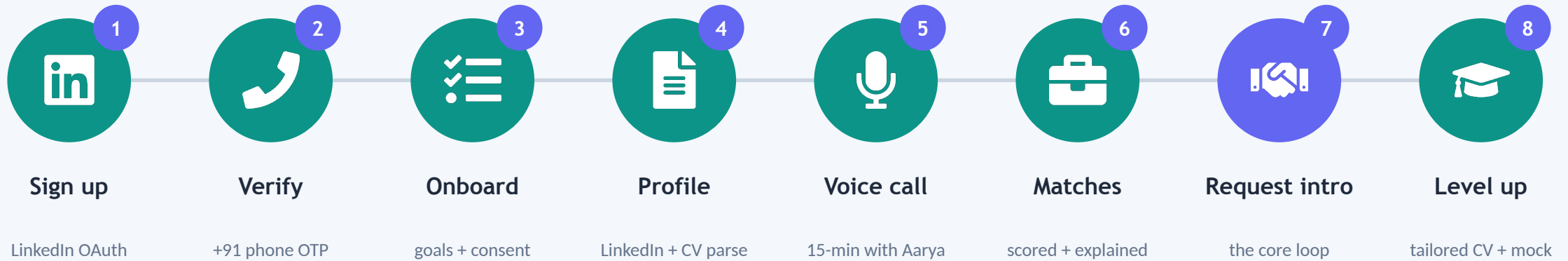


Recruiter workspace

roles
role_versions
role_pipeline
recruiter_searches

20 SQL migrations · pgvector HNSW (cosine) on every *_embedding · Row-Level Security on every table · soft-delete on PII

From LinkedIn click to a warm intro



What makes it different

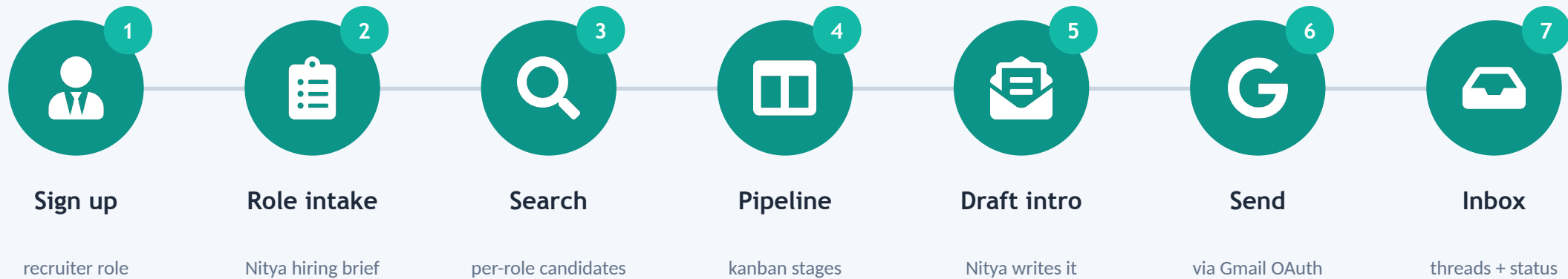
Zero-form onboarding

— profile is built from LinkedIn + CV + voice, not typed.

Honest, explained matches

— every role shows a score and why it fits, India-only.

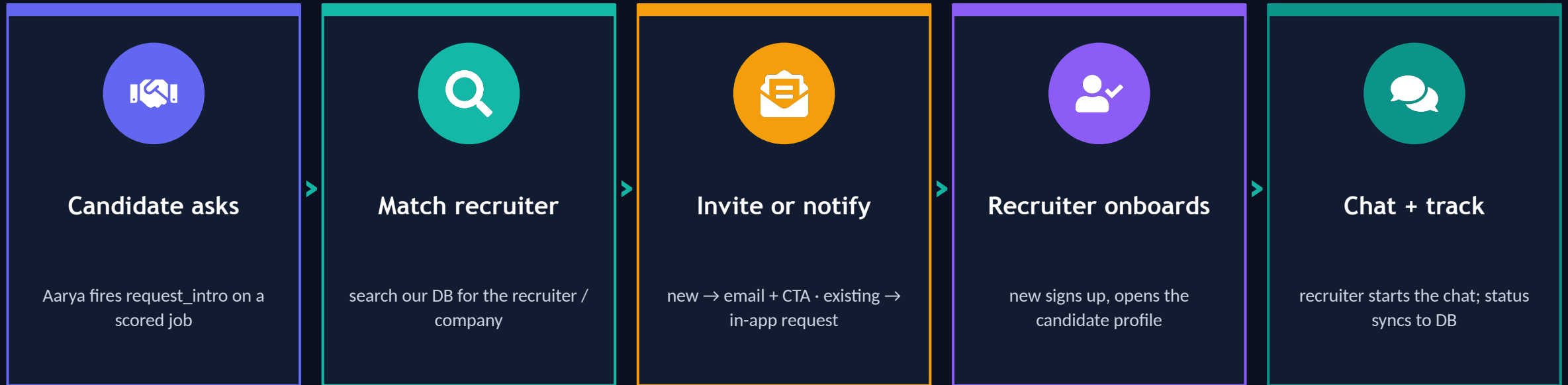
From open role to candidate intros



Recruiters are pulled in on demand

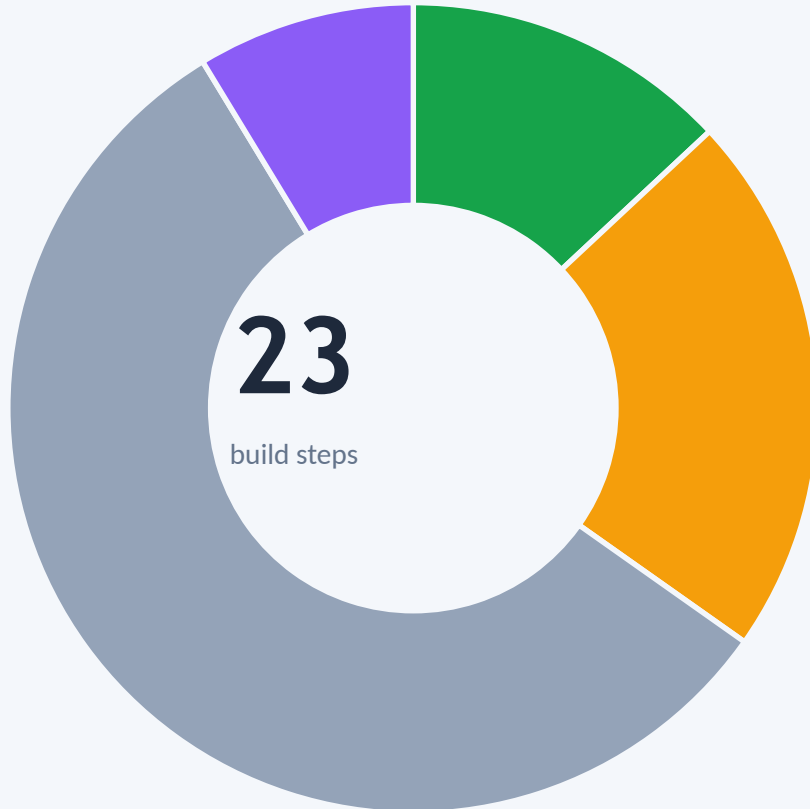
When a candidate requests an intro, the platform looks the recruiter up in our database. Already on Hireloop? They get an in-app intro request. New to us? They get a one-click email invite to sign up, see the candidate, and start the chat. The database — never agent-to-agent calls — keeps both sides in sync. No cold email, ever.

Intro handshake — the MVP-critical path



DB-driven — recruiters are pulled in on demand: a new recruiter gets a one-click email invite, an existing one an in-app request. No cold email, no agent-to-agent RPC.

Where the build stands today



3 Done

LinkedIn signup · onboarding wizard · résumé upload + parse

5 Improving

Phone OTP (verify / alt) · Aarya text + voice · matching · match feed

13 Pending

S07 job ingestion (Apify ready) · S10–S21 (intros, recruiter side, admin, SEO)

2 Planned

S22 deploy (Vercel + Supabase) · S23 payments (v2)

Build status — all 23 steps, in order



Done



Improving



Pending



Planned

 S01 LinkedIn signup	 S09 Job match feed UI	 S17 Pipeline kanban
 S02 Phone OTP — verify / alt	 S10 Hiring-manager enrichment	 S18 Recruiter inbox + Nitya
 S03 Onboarding wizard	 S11 Gmail intros	 S19 Admin panel + DPDP
 S04 Aarya text chat	 S12 Intros inbox	 S20 Transactional email
 S05 Aarya voice session	 S13 Mock interview	 S21 Programmatic SEO site
 S06 Résumé upload + parse	 S14 Tailored résumé per JD	 S22 Deploy — Vercel + Supabase
 S07 Job ingestion (Apify)	 S15 WhatsApp notify	 S23 Payments (v2)
 S08 Embeddings + matching	 S16 Recruiter signup + roles	

In progress, pending & deferred

EXTERNAL DEPENDENCIES

Step	Needs	Status
S07 Job ingestion	APIFY_TOKEN	In hand — wiring & running now
S02 Phone OTP	Twilio / MSG91	Implemented but flaky — verify / replace
S11 Gmail intros	Google OAuth app	Pending app verification
S20 Transactional email	SendGrid key	Pending
S15 WhatsApp notify	MSG91 + Meta	Pending (~2-wk template approval)



S10-S21 — Pending

HM enrichment · intros + inbox · mock interview · tailored résumé · recruiter signup · pipeline · Nitya inbox · admin / DPDP · SEO.

S22 Deploy → Vercel (app + web) + Supabase + CI/CD

S23 Payments → deferred to v2 (manual tracking)

Keys & integrations to sort before production

In hand: APIFY_TOKEN

Needed: GOOGLE_CLIENT_ID/SECRET · SARVAM_API_KEY · SENDGRID_API_KEY · SECRET_KEY · SERVICE_SECRET

Flaky / revisit: Twilio + MSG91 OTP (verify or replace) · MSG91 WhatsApp (Meta approval) · NeverBounce (not implemented yet)

How much load the planned setup can take

Component	Planned sizing	Comfortable MVP load*	Real bottleneck / scaling lever
Next.js app (Vercel)	Serverless + edge CDN	Auto-scales to 100s of users	Vercel plan limits · function duration
FastAPI (async)	Python 3.12 · async workers	~200–500 req/s I/O-bound	Horizontal: more instances behind LB
Supabase Postgres	Pro tier + pooler	1000s light queries/s	Connection pool size; upgrade tier
pgvector search	HNSW, cosine	<10 ms over 100k–1M vectors	Index RAM; partition by region
Aarya voice (Deepgram→Sarvam)	Pay-as-you-go	~tens of concurrent calls	STT/TTS stream concurrency quota
Agent chat (OpenRouter)	Claude via API	Rate-limit bound	Token rate limit + \$ per turn
Job ingest (Apify)	Scheduled pg_cron	Batch, off-peak	Apify compute units (not user-facing)

* Planning estimates for an MVP / pilot (≈ up to 1,000 active candidates, tens of concurrent voice calls). Not yet validated under load — S22 deploy pending. The Next.js app auto-scales on Vercel (serverless); the near-term limits are external-service quotas (voice STT/TTS concurrency, OpenRouter rate + cost) and the Supabase tier (connections, Realtime sockets).

Tools, costs & renewals

Tool	What it's for	Plan / tier	Cost	Renews / expires
Supabase	Postgres · Auth · Storage · Realtime	Pro (annual)	\$55 / yr	Yearly
Vercel	App + web hosting (edge CDN)	—	—	Not bought yet
Deepgram	Voice STT / TTS — in use now	Pay-as-you-go	—	In use
Sarvam AI	Voice STT / TTS — planned (Indian langs)	Planned	—	Not bought yet
OpenRouter	Agent LLM (Claude) — runtime	Credits	\$20 top-up	On recharge
Apify	LinkedIn + job scrapers	Pay-as-you-go	\$22 / 3 mo	Every 3 months
Affinda	Résumé parsing	—	—	Not bought yet
Twilio	Phone OTP / SMS	—	—	Not bought yet
MSG91	SMS / WhatsApp	—	—	Not bought yet
NeverBounce	Email verification (planned)	—	—	Not bought yet
Gmail OAuth	Recruiter intro sending	Free	Free	—
SendGrid	Transactional email	—	—	Not bought yet
Domain (hireloop.in)	Registrar / DNS	—	—	Not bought yet
Claude (Anthropic)	Dev / build assistant	Pro	\$20 / mo	Monthly
Cursor	AI code editor — dev tool	Pro	\$55 / yr	Yearly

Runtime spend is usage-based (OpenRouter, Deepgram → Sarvam, Apify, Twilio, MSG91). Claude + Cursor are build-time dev tools. Rows marked “Not bought yet” aren't purchased.

Risks & must-fix items



Rotate exposed credentials

Live API credentials are currently committed inside a planning doc (PHASE_TRACKER.md). Rotate them and move every secret into Vercel / Supabase env (.env) before any deploy.



Limited end-to-end testing

The built steps work in dev, but the full journeys aren't validated e2e with live keys. Stand up staging and walk both journeys before launch.



Deployment not started (\$22)

Vercel (app + web), Supabase project hardening, and GitHub Actions CI/CD still to be provisioned and wired to the custom domains.



Flaky integrations + approvals

Twilio + MSG91 OTP are unreliable — verify or pick an alternative. MSG91 WhatsApp needs Meta approval (~2 wks); Google OAuth needs verification. Start early.

Roadmap & tracking in Linear

1

Procure keys

Google OAuth, SendGrid, Sarvam (Apify ✓)

2

E2E test staging

Walk candidate + recruiter journeys

3

Deploy (S22)

Vercel + Supabase + CI/CD

4

Pilot launch

Onboard first candidates & recruiters



Tracking model for Linear

Epics → journeys. Three epics — Candidate journey, Recruiter journey, and Platform & Infra.

Issues → steps. Each S-step (S01–S23) becomes one issue, labelled done / blocked-on-key / not-started / deferred.

Sub-tasks → the per-step checklist (test with real keys, DB check) already written in PHASE_TRACKER.md.

Labels → area:candidate · area:recruiter · area:infra · blocked:key for filtered views and burn-down.



HIRELOOP

Two AI agents. One candidate graph.

8 of 23 steps built · architecture set · intro loop + keys + deploy next.